

ParkZone – Vehicle Parking App

Author

Name: Om Manish Makadia

Roll Number: 23f3001593

Email ID: 23f3001593@ds.study.iitm.ac.in

About: I am an enthusiastic learner in Artificial Intelligence and Data Science, currently pursuing a diploma level in online BS degree in Data Science & Applications from IIT Madras, alongside an offline BE degree in Artificial Intelligence & Machine Learning from BMS College of Engineering, Bengaluru.

Description

I have to develop a multi-user web application to manage 4-wheeler parking lots and vehicle reservations with role-based access for administrators and users. Administrators should be able to create, edit, delete, and monitor parking lots, while users can register, log in, and book or release parking spots. The system must automatically allocate available spots, support custom pricing per lot, and maintain reservation history.

The application is named **ParkZone**, representing a dedicated zone where users can park their 4-wheeler vehicles with ease and efficiency.

AI/LLM Use:

Usage Percentage – 8% (Bootstrap / CSS)

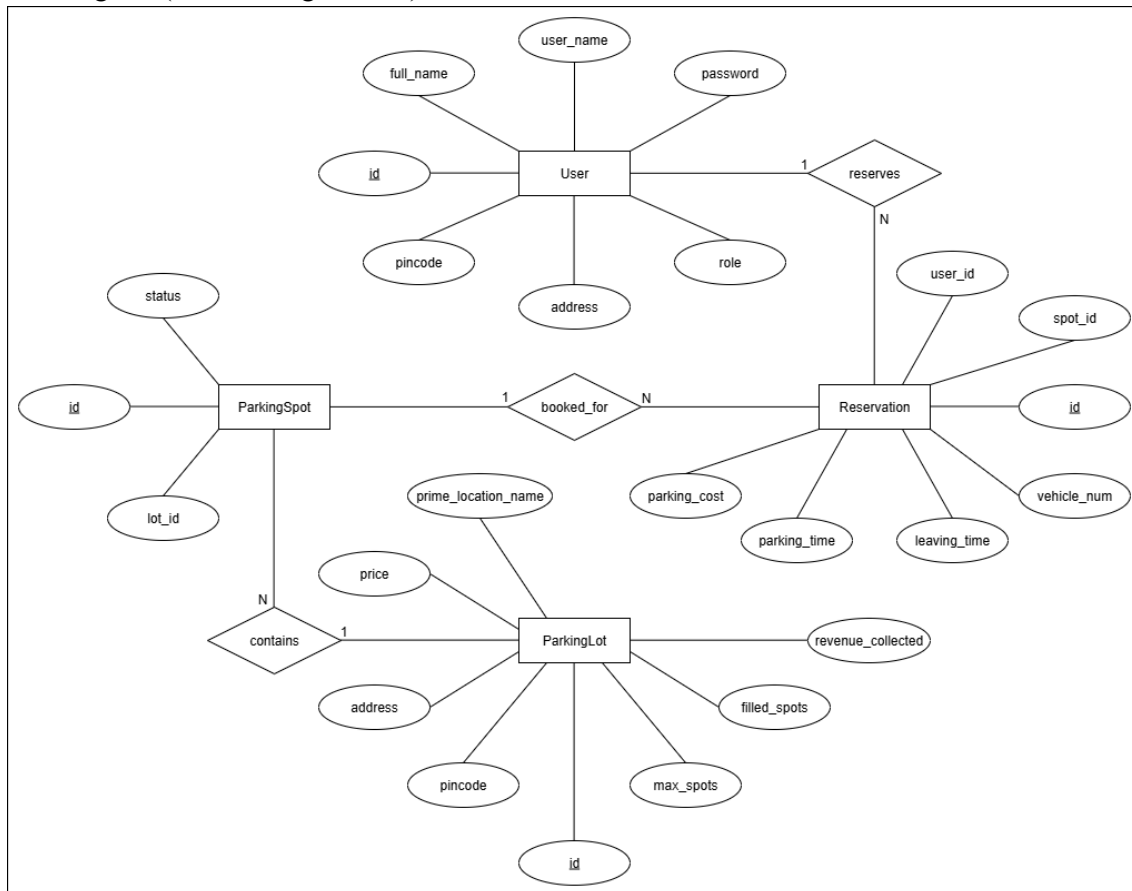
I have used AI/LLM (ChatGPT) in this project primarily as a learning and guidance tool to understand concepts, clarify doubts, and explore best practices. No code was copied from the AI; all implementation is original and written by me, based on my understanding. The AI's role was purely educational to help me understand the concepts. The only area of direct assistance was in custom CSS and Bootstrap, where I gave clear design requirements and later understood the purpose of each suggested element. Overall, AI/LLM was used to support conceptual understanding and guide implementation decisions, ensuring that all code was developed independently with full ownership, understanding and clarity.

Technologies Used

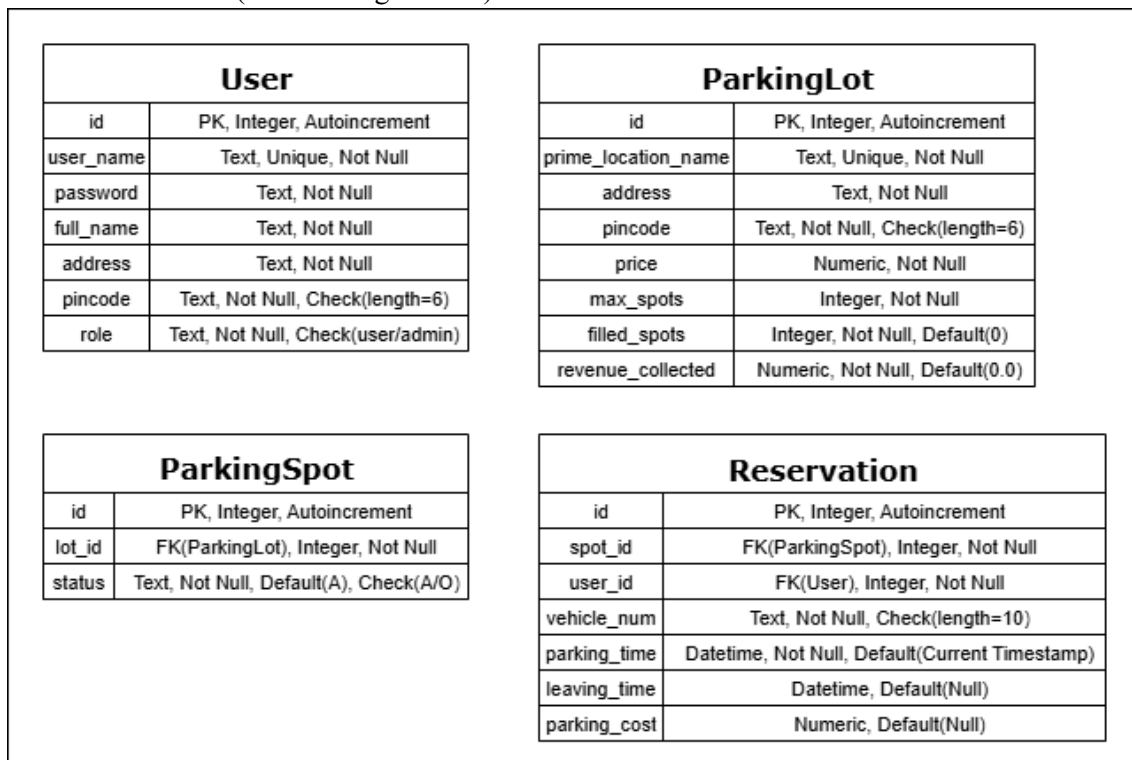
Technology	Purpose
HTML	To structure and display content on the web pages.
Jinja	To dynamically render HTML templates with Python variables using Flask.
CSS (Custom)	To apply minor custom styles and overrides not covered by Bootstrap.
Bootstrap	To build responsive layouts using pre-designed UI components.
JavaScript (Basic)	To display alert messages for form validation errors and API failures.
SQLite	To store application data in a database.
Flask	To create the main backend web application and handle HTTP requests.
Flask-SQLAlchemy	To interact with the SQLite database using Python ORM models.
Flask-RESTful	To build modular REST API endpoints for various operations.
Flask-Login	To manage user authentication, login sessions, and access control.

DB Schema Design

ER Diagram (drawn using draw.io):



Relational Schema (drawn using draw.io):



Reasons behind the Database Design:

- Merged Admin and User in a Single Table (User): To avoid duplication and ensure a normalized design, both admin and regular users are stored in a single User table. The role attribute is used to distinguish between them ('user' or 'admin'). Since an admin is essentially a privileged user (superuser), there was no need for a separate table. This design simplifies authentication and authorization logic.
- Tracking Revenue and Spot Availability: The ParkingLot table includes revenue_collected, max_spots, and filled_spots attributes to allow efficient dashboard analytics, such as utilization and earnings tracking, without needing to compute them repeatedly from historical reservations.
- ParkingSpot Linked to ParkingLot via Foreign Key: Each parking spot is associated with one parking lot through a foreign key (lot_id). This allows for dynamic scalability of parking lots and keeps the data consistent. It also helps in managing and querying spot availability on a per-lot basis.
- Reservation Records Map Users to Parking Spots: The Reservation table acts as a link between User and ParkingSpot. This design ensures that the many-to-many relationship between users and parking spots over time is accurately captured. A user can have multiple reservations, and each spot can be reserved multiple times (historically), making this a classical many-to-many mapping resolved via a separate table.
- Data Integrity via Constraints: Check constraints are used to maintain clean and valid data across the system, and to enforce domain restrictions, such as,
 - pincode must be exactly 6 digits.
 - vehicle_num must be 10 characters long.
 - role must be either 'user' or 'admin'.
 - status of a parking spot must be 'A' (Available) or 'O' (Occupied).

API Design

APIs are designed using the Flask-RESTful extension to modularize and structure backend functionality into four main groups: Authentication, User, Parking Lot, and Reservation.

- Authentication APIs: These are implemented to allow user login and new user registration using POST methods with input validation.
- User APIs: These APIs handles user profile management, dashboards, and user listing. These include endpoints for fetching and updating user profiles, retrieving dashboards for users and admin, and listing registered users with optional search functionality.
- Parking Lot APIs: These enables admin to create, read, update, and delete parking lots. They also support viewing currently occupied spots in a specific lot with real-time details.
- Reservation APIs: These APIs allow users to book and release parking spots and view their booking history. Admins can also view all completed reservations. Spot allocation is handled automatically based on availability, and cost is calculated at release time.

All APIs follow REST principles with appropriate use of HTTP methods (GET, POST, PUT, DELETE) and error handling via custom exceptions. Input parsing is done using reqparse, and responses are structured using @marshal_with and defined output fields for consistent formatting.

Architecture and Features

The project is organized into clearly defined folders following a modular structure, separating concerns based on functionality. The controllers folder contains the route handlers split into main.py (authentication and common features), admin.py (admin-specific features), and user.py (user-specific features). The models folder includes models.py for defining database models using SQLAlchemy and the actual SQLite database file. All API-related logic is placed under the resources folder, structured into separate files for authentication, user, reservation, and parking lot APIs. The templates folder holds HTML templates grouped into subfolders: main for login and homepage, admin for admin-related views, and user for user dashboards and pages. Static files like CSS (styles.css), images (logo.png), and JavaScript (error-handler.js) are placed in the static folder. The application is initialized in app.py, which wires together the database, controllers, login system, and APIs.

The application includes a range of essential and additional features to ensure security, usability, and functionality:

- User Session Management: All routes are protected using Flask-Login, ensuring only authenticated users can access restricted pages.
- Role-Based Access Control: Decorators like @admin_required and @user_required are used to restrict access based on roles (admin or user).
- Security Measures: A secure SECRET_KEY is generated using secrets.token_hex(32) to protect sessions and cookies.
- Search Functionality: Admins can search for registered users and completed reservations. Users can search within their reservation history using keywords.
- Form Validation: All user inputs in forms are validated on both client and server sides for required fields, correct formats, and appropriate values.
- User Feedback via Alerts: JavaScript is used to display user-friendly error alerts, enhancing the interface and guiding users through actions.
- Profile Management: Both admins and users can update their profile details through dedicated interfaces.
- Account Deletion: Users are allowed to delete their account only if there are no ongoing reservations linked to them, ensuring data consistency.

Video

https://drive.google.com/file/d/17-tRI9d1UWLRTt8GpsOCaLnaurthnP2/view?usp=drive_link